

¿Qué es un patrón de diseño en el desarrollo de software?

Un patrón de diseño es una solución comprobada y recurrente a problemas comunes que surgen en el diseño de software. No es una porción específica de código que se copia y pega, sino un concepto general o un "plano" para resolver un problema particular. Sirven como un lenguaje común entre desarrolladores, facilitando la comunicación y promoviendo el uso de buenas prácticas de diseño orientado a objetos.

¿Por qué es importante aprender sobre patrones de diseño?

Aprender sobre patrones de diseño es crucial porque proporcionan un conjunto de herramientas de soluciones probadas para desafíos habituales en el diseño de software. Aunque se podrían implementar patrones sin saberlo, conocerlos explícitamente enseña a resolver problemas utilizando principios de diseño orientado a objetos, mejora la comunicación en equipo al establecer un lenguaje común (ej., "usa un Singleton para eso"), y fomenta el desarrollo de sistemas más eficientes, mantenibles y adaptables.

¿Cómo se clasifican los patrones en el diseño de software?

Los patrones se clasifican según su complejidad, nivel de detalle y escala de aplicabilidad. Los más básicos y de bajo nivel son los *Idioms*, aplicados a un único lenguaje de programación. Los *patrones de diseño* son soluciones probadas para problemas de diseño de software. Los más universales y de alto nivel son los *patrones de arquitectura*, que pueden usarse para diseñar la arquitectura de una aplicación completa en casi cualquier lenguaje.

Dentro de los patrones de diseño, se pueden clasificar por su propósito en tres tipos:

- **Patrones creacionales:** Se encargan de la creación de objetos, aumentando la flexibilidad y reutilización del código existente.
- **Patrones estructurales:** Explican cómo ensamblar objetos y clases en estructuras más grandes, manteniendo flexibilidad y eficiencia.
- **Patrones de comportamiento:** Gestionan la comunicación efectiva y la asignación de responsabilidades entre objetos.

¿Cuáles son algunos ejemplos de patrones de diseño creacionales comúnmente utilizados en Java?

En Java, los patrones de diseño creacionales incluyen:

1. **Factory Method (Método Fábrica):** Crea objetos sin especificar la clase exacta, delegando la creación a subclases.
2. **Abstract Factory (Fábrica Abstracta):** Proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.
3. **Constructor (Builder):** Construye objetos complejos paso a paso, separando la construcción de la representación para crear diferentes representaciones con el mismo proceso.

4. **Singleton:** Garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.

¿Qué es el patrón Singleton y para qué se utiliza?

El patrón Singleton garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a esta. Se utiliza cuando se necesita controlar estrictamente la cantidad de instancias de una clase, por ejemplo, para gestionar recursos compartidos, configuraciones o, como en el caso de la actividad, un DiscountManager que aplica descuentos de manera centralizada. Para implementarlo, se recomienda ocultar el constructor, declarar un método getInstance() como estático, y asegurar la atomicidad en entornos concurrentes.

¿Cuál es la estructura clave para implementar el patrón Singleton?

Para implementar el patrón Singleton, se deben seguir los siguientes pasos y características:

- **Constructor privado:** La clase Singleton debe tener un constructor privado para evitar que se creen instancias directamente desde fuera de la clase.
- **Instancia estática y privada:** Dentro de la clase, se declara una instancia privada, estática y final de la propia clase (ej., private static final Singleton instance = new Singleton();).
- **Método estático getInstance():** Se crea un método público y estático getInstance() que devuelve la única instancia de la clase. Este método es el punto de acceso global para obtener la instancia.

¿Qué entornos de desarrollo y herramientas son recomendados para

trabajar con patrones de diseño en Java?

Para trabajar con patrones de diseño en Java y, específicamente, para la actividad propuesta, se recomienda utilizar:

- **Java Development Kit (JDK):** Es el entorno de desarrollo necesario para compilar y ejecutar aplicaciones Java.
- **Apache NetBeans:** Es un entorno de desarrollo integrado (IDE) que facilita la codificación, depuración y despliegue de aplicaciones Java. Se sugiere descargar la versión 18 de Apache NetBeans por ser la más estable.

En un sistema de pedidos online, ¿cómo se aplicaría el patrón Singleton

para gestionar descuentos?

En un sistema de pedidos online, el patrón Singleton se aplicaría para crear una instancia única de una clase DiscountManager. Esta clase DiscountManager sería responsable de aplicar los diferentes descuentos y promociones a los productos. Al ser un Singleton, se asegura que solo exista una instancia de DiscountManager en toda la aplicación, lo que garantiza que todos los productos y

pedidos accedan al mismo gestor de descuentos y que las políticas de descuento se apliquen de manera consistente y centralizada desde cualquier parte del sistema. Se accedería a esta instancia única a través de un método estático getInstance()).